Orbyte — Complete Demo Script

Live site: https://orbyte-0dw7.onrender.com/ **Repository:** https://github.com/AlyssaWoe/UMvengers-Orbyte **Run time:** 10 minutes (a 3-minute lightning version is in Appendix C) **Demo files:** `demo/Orbyte-Demo-Invoice-INV-2026-8802.pdf` (primary), `INV-2026-8803.pdf` (spare), `INV-2026-8801.pdf` (already used once on the live instance — do not reuse it there)

Before you start (10 minutes before)

1. Open the live URL in Chrome and let the 3D park finish loading. On Render's free tier the first request after idle can take 30–60 seconds; load it once, then leave the tab open.
2. Confirm the top bar shows **Upload Bill** and the hamburger menu. That means the backend is awake.
3. Put the `demo` folder on the Desktop so the file picker opens in the right place.
4. Do **not** open the RunPod settings panel during the demo. The endpoint is not linked, and the panel honestly reports **Local / Not Linked**. It is a strength if you mention it, a distraction if you stumble into it.
5. Upload each invoice ID **once per running instance**. A repeat upload is correctly rejected as a duplicate, but the dossier panel keeps the previous invoice on screen, which looks like a bug to an audience.
6. Check whether the pending fixes are live. Open Agent 5: if the governance note says **$5,000**, the threshold copy fix is not deployed — do not read that sentence aloud, and rely on Agent 7's $4,000 gate instead. Run the acceptance test once: if Stage 3 shows **17%**, the classifier demo-message fix is not deployed — skip that figure as described below.
7. Numbers on screen will differ from this script if the backend has restarted or if an earlier test ran. Read the screen, not the script.

Two honest facts to keep in your pocket

- Agent 1, the Executive Overview, and the Agent 7 acceptance test are **live backend calls**.
- The Agent 2, 3, 4, 5 and 6 workstations display **curated demo datasets** that visualise those engines' output. The engines themselves are real and are exercised end-to-end by the Agent 7 acceptance test. Do not describe the static panels as live API calls.

0:00 — Opening (30 seconds)

SAY:

> "Small and medium businesses lose cash not because customers refuse to pay, but because nobody follows up on time. Orbyte is an accounts-receivable agent that ingests an invoice, watches the due date, understands the customer's reply, drafts the follow-up, reconciles the payment, and stops for human approval whenever the amount is large or the facts are ambiguous."

> "This is the deployed site, not a video. Every agent you will see runs in this browser session."

0:30 — The 3D campus (90 seconds)

DO: Leave the campus on screen. Drag once to orbit slightly, then press **W** for two seconds so the avatar walks.

SAY:

> "The campus is a single React Three Fiber canvas: a ring headquarters modelled on Apple Park, seven voxel agents standing at compass points inside the courtyard, and a playable avatar. It is rendered in WebGL in the browser, not an embedded video."

> "Each workstation belongs to one agent. The ring is 48 metres outer, 36 metres inner; the seven agents sit on a 26-metre circle, with the Orchestrator at the centre. The camera is a fixed, damped overview at about 72 metres, with orbit clamped so the view never spins or makes anyone dizzy."

DO: Walk toward **Payment Monitoring** (Agent 2) with **W/A/S/D**. When the prompt appears at the bottom, press **E**.

SAY:

> "Proximity is part of the interface: within about five metres the avatar surfaces a talk prompt, and E opens the agent's card. Clicking any agent does the same thing."

DO: In the modal, press **Open Staff Dashboard** →, then **Back to 3D Campus** to return. Reset with 🔄 **Steady Overview**.

> Full technical breakdown of the 3D layer is in Appendix A.

2:00 — Agent 1: upload a real invoice (2 minutes)

DO: Click 📄 **Upload Bill** in the top bar (this is a shortcut straight to Agent 1). Then click the dashed upload box and choose `Orbyte-Demo-Invoice-INV-2026-8802.pdf`.

SAY while it loads:

> "Agent 1 is live. It does not trust the file extension. A PDF goes through pypdf for its text layer; an image goes to OCR; RunPod is a fallback when it is configured. This PDF is a generated text-layer invoice from a fictional supplier, Straits Marine Supply."

POINT AT after about two seconds:

- The green banner: **Extracted successfully via Docling Layout & Table Engine (PyPDF Native)**.
- The left panel: the raw text stream the model actually received.
- The right panel: **INV-2026-8802**, **Straits Marine Supply**, due **2026-10-18**, two line items, **$2,350.00**, **VALIDATED**, **98.5% confidence**.

SAY:

> "The extraction is deterministic, and so is the validation. The parser pulls the invoice ID, client, dates, currency and line items, then Pydantic enforces the schema. The line items must sum to the printed

> total within one dollar. If they do not, the invoice is not accepted: it is routed to data review with the discrepancy recorded."

> "This is the core design rule of Orbyte: the language model drafts, but deterministic code decides. The invoice is now written to the ledger and to the append-only audit trail."

IF THE UPLOAD FAILS: use `INV-2026-8803.pdf`. If that also fails, the backend is asleep — open the **3D campus** and retry once; if it still fails, continue the demo from the Executive Overview and describe Agent 1 from the pre-loaded `INV-2026-0891` dossier.

DO NOT click **Execute Docling & Pydantic Verification** after a successful upload. The invoice already exists, so the duplicate guard fires and the button appears to do nothing. It is only needed when you paste text manually.

4:00 — Executive Overview (45 seconds)

DO: Hamburger menu → 📊 **Executive Receivables Overview**.

SAY:

> "Every agent writes to one authoritative ledger. This is the single source of truth: total outstanding, overdue accounts, reconciled settlements, and the state of every invoice."

POINT AT:

> "The RunPod pill reads **Local / Not Linked**, and that is deliberate. The hosted GPU worker and its client are implemented in the repository, but no endpoint has been provisioned, so the app falls back to its local deterministic engines instead of pretending a model is connected."

DO: Scroll once to show the invoice table, then go back.

4:45 — Agent 2: aging and surveillance (45 seconds)

DO: Hamburger → **Agent 2: Payment Monitoring & Aging**.

SAY:

> "Agent 2 is the clock. It compares every due date with today, buckets accounts into overdue, upcoming and paid, and applies a quiet-hours policy so nobody is chased at two in the morning."

DO: Click the **Overdue**, **Upcoming**, **Paid** tabs. Then click one row to open the client dossier.

> "The dossier gives the officer the contact, the aging, the balance and the last action in one place."

DO: Close the dossier with the ×.

5:30 — Agent 3: Laya decision and routing (45 seconds)

DO: Hamburger → **Agent 3: Laya Decision & Routing**. Click the **Billing Dispute** scenario chip.

POINT AT:

- Label **INVOICE DISPUTE**, **94.1% confident**
- Route **ROUTE_TO_HUMAN_AR_OFFICER**
- The six-candidate score distribution

SAY:

> "Laya is a non-autoregressive scorer: it scores six fixed candidate labels in one pass instead of generating free text. The top label is only acted on if its calibrated confidence clears 0.75. Here a dispute is confidently recognised, and the confidence is exactly why the route is a human officer, not an automated reminder."

DO: Click the **Ambiguous Message** chip.

> "And here is the safety property that matters: when the message is 'hello noted ok', confidence collapses to 41%, the threshold is not met, and the workflow falls back to human triage. The system would rather escalate than guess."

6:15 — Agent 4: reading the customer's reply (45 seconds)

DO: Hamburger → **Agent 4: DeepSeek Reply Understanding**. Keep the first conversation selected.

POINT AT the red banner:

> "The customer promises to wire $4,850 next Tuesday, 12 November. The stated weekday and the calendar date disagree: 12 November 2026 is a Thursday. This is the P0 date-versus-weekday conflict guard. Both statements are shown to the officer with their sources; the system does not silently rewrite the date, and it does not schedule a reminder against a date it cannot trust."

DO: Select the second conversation (Clean Promise) to show the non-conflict state, then the third (Billing Dispute) to show the dispute path.

> "Same pipeline, three different outcomes: a clean commitment pauses reminders, a conflict escalates for confirmation, a dispute freezes collection and opens a resolution task."

> "The adversarial screen also runs before any model sees the text. A message that tries to instruct the agent is flagged, and the model is skipped rather than trusted."

7:00 — Agent 5: the drafted follow-up (30 seconds)

DO: Hamburger → **Agent 5: Collection Strategy & Follow-up**.

SAY:

> "Agent 5 turns the structured facts into a policy-compliant follow-up. Tone moves from friendly to formal to urgent as the aging bracket worsens, and the channel can be email, WhatsApp or SMS."

DO: Click **Urgent** and **WhatsApp** to show the selection states. Do not claim the draft body regenerates; it is the policy example.

POINT AT the approval note:

> "Anything above four thousand dollars never leaves the building automatically. It becomes an approval task."

7:30 — Agent 7: the full acceptance test (90 seconds)

DO: Hamburger → **Agent 7: Orchestrator, Approval & Audit**. Click ▶️ **Execute Full Acceptance Test**. Wait about five seconds.

SAY while it runs:

> "This is the whole company in one request: a four-thousand-eight-hundred-and-fifty-dollar overdue invoice, eight stages, seven agents, one traversal."

WALK DOWN THE RESULTS:

> "Stage 1, ingestion and schema validation. Stage 2, deterministic aging: fourteen days overdue. Stage 3, Laya classifies the reply as a payment promise above the calibrated threshold and routes it to Agent 4. Stage 4, the promised date is extracted and checked against the weekday; no conflict on this run. Stage 5, the follow-up draft is produced."

POINT AT STAGE 6:

> "Stage 6 is the governance gate. The draft is blocked from auto-send because the balance exceeds the four-thousand-dollar threshold. This is backend policy, not a frontend warning. Stage 6 is the proof that the model is never the authority."

POINT AT STAGES 7 AND 8:

> "Stage 7, a twenty-five-hundred-dollar partial deposit is matched deterministically and the balance drops to twenty-three-fifty. Stage 8, Agent 2 resyncs against the new authoritative balance and Agent 7 appends the audit record. All eight stages passed."

OPTIONAL: tick **Test P0 Date Conflict** and run again. This uses a message that says "next Tuesday, 12 November". Stage 3 still classifies a payment promise, and Stage 4 flips to **FLAGGED_FOR_HUMAN_REVIEW** with Tuesday versus Thursday. That contrast is the strongest single moment in the demo if you have 30 extra seconds.

> **Deployment note:** if Stage 3 shows **17%** instead of **78%**, this instance has not picked up the fixed acceptance-test message yet. The stage still completes, but do not read the confidence aloud; move

> straight to Stage 4 and say the classifier routes low-confidence replies to human review by design.

9:00 — Closing (30 seconds)

SAY:

> "Orbyte is not a chatbot that sends emails. It is a deterministic receivables workflow with language models inside it: one ledger, seven agents, an explicit human gate above four thousand dollars, and an audit trail that a finance team can defend. The 3D campus is the navigation layer; the staff dashboards are where the work happens; the backend is where the policy lives."

> "The code, the RunPod worker and this script are all in the UMvengers-Orbyte repository."

Appendix A — How the 3D campus actually runs

Stack. React 18 + TypeScript, `@react-three/fiber` for the render loop, `@react-three/drei` for `Sky`, `Html` and `OrbitControls`, Three.js underneath. One `<Canvas>` with shadows, 45° field of view, near 0.1, far 500.

Scene graph (all in `frontend/park/src/components/3d/`):

Component	What it does
`ParkScene.tsx`	Owns the canvas and composes the whole campus
`Sky` + `DaylightLighting`	Sky shader with sun at `[45, 65, 35]`; directional sun plus fill light, shadow maps on
`ParkGround.tsx`	Terrain, berms, entrance plaza
`CurvedPath.tsx`	Concentric and radial pedestrian paths
`CircularHeadquarters.tsx`	Ring HQ: 48 m outer radius, 36 m inner, 5.6 m tall, 4 floors
`TreeCluster.tsx`	Procedural tree clusters, placements memoised once at mount
`Pond.tsx`, `Garden.tsx`, `Bench.tsx`, `Clouds.tsx`	Scene dressing, all procedural geometry
`VoxelAgent.tsx`	One agent per department, seven instances
`PlayerAvatar.tsx`	The playable voxel character
`CameraController.tsx`	Fixed, damped orbit rig

Agents. Each `VoxelAgent` is built from boxes at Minecraft proportions — 0.48 m head, 0.54 × 0.7 × 0.28 torso — with a glass desk, dual monitors tinted in the department colour, a coffee mug and an office chair. Two arm groups rotate on a sine wave to produce a typing idle animation; the head sways slightly. A selection ring appears on hover or selection. The floating name pill is **DOM**, not 3D: drei's `<Html>` with

`distanceFactor={18}`, which keeps the labels crisp at any zoom. Clicking the group calls `onSelect(agent)` and opens the modal.

Positions. Agent 1 south `[0,0,26]`, Agent 2 south-east `[24,0,12]`, Agent 3 north-east `[24,0,-12]`, Agent 4 north `[0,0,-26]`, Agent 5 north-west `[-24,0,-12]`, Agent 6 south-west `[-24,0,12]`, Agent 7 at the centre `[0,0,0]`. Each faces the centre via a fixed Y rotation.

Player movement. Input is world-space, not camera-relative: W is always −Z regardless of orbit. Velocity is lerped toward the target direction for acceleration and damping. Walk speed is 8.5 units/s, sprint with Shift is 14. The character rotates toward the movement vector with a shortest-angle lerp. Limbs swing on a sine cycle while moving and ease back to rest when idle. Position is clamped to a 34 m radius so the avatar cannot walk through the headquarters.

Proximity. Every frame the player compares its position with the seven agent positions. Within 5.2 m the HUD shows the interaction prompt; pressing E opens the agent if within 5.5 m. Clicking works at any distance.

Camera. Position `[0, 72, 68]`, target `[0, 0, 0]`, `enableDamping` at 0.08, distance clamped 20–160, polar angle clamped 0.1π–0.45π, `autoRotate` off. The **Steady Overview** button resets camera and avatar. It is intentionally a fixed, high three-quarter view: the user asked for no dizziness, so there is no auto-rotation and no follow-cam.

Performance and navigation. One canvas, instanced foliage, no post-processing. Opening any staff dashboard sets the canvas container to `display: none` rather than unmounting the whole app, so returning to campus is instant and the WebGL context is not rebuilt.

Appendix B — How each agent runs

Agent 1 — Data Ingestion & Validation

Live. A file arrives at `POST /api/upload`. Extraction order: local first (pypdf for PDFs, Windows Media OCR for images, plain-text decode for text files with a NUL-byte guard that refuses binary), then the RunPod OCR worker if configured and local extraction produced nothing. Regex extracts invoice ID, email, client, two dates, currency and amounts. Line items are parsed and their sum is compared with the printed total; a discrepancy over $1 is an error. Pydantic builds a `ValidatedInvoice`; a duplicate ID is rejected before insertion. The invoice, initial state and an audit event are written to the store.

Agent 2 — Payment Monitoring & Aging

Engine real, workstation illustrative. `evaluate_invoice_status` compares due date to today, computes days overdue, checks quiet hours and dispute holds, and decides follow-up eligibility. The dashboard shows representative buckets: four overdue accounts, one upcoming, one settled, each with a contact dossier.

Agent 3 — Laya Decision & Routing

Engine real, workstation illustrative. Six fixed labels are scored from indicator phrases, then normalised into calibrated probabilities. Threshold 0.75; below it the route is **Safe Fallback / Human Review** regardless of the top label. Above it, payment promise routes to Agent 4, already-paid routes to Agents 4 and 6, dispute routes to a human AR officer, extension routes to Agent 5. The dashboard's four chips replay representative inputs.

Agent 4 — DeepSeek Reply Understanding

Engine real, workstation illustrative. Deterministic extraction first: intent, claimed payment, promised date, stated weekday, actual weekday, dispute reason. A prompt-injection screen runs before anything else and blocks model enrichment when it fires. The P0 guard compares the stated weekday with the real calendar weekday of the parsed date; a mismatch escalates instead of scheduling. When RunPod is configured, a model may add a summary and a suggested intent, but only as annotations — it cannot clear a block or change the deterministic facts. The dashboard shows four conversations: date conflict, clean promise, dispute, already-paid claim.

Agent 5 — Collection Strategy

Engine real, workstation illustrative. The policy engine picks a tone from the aging bracket, drafts the follow-up against the authoritative balance, and marks any balance above $4,000 as requiring human approval. The dashboard exposes tone and channel selection with a representative draft.

Agent 6 — Payment Reconciliation

Engine real, workstation illustrative. Deterministic three-way matching between bank transaction, open receivable and any customer claim. Exact matches close the invoice; partial deposits reduce the balance and record a payment entry with previous and remaining balance; unmatched wires go to a suspense hold rather than being forced onto an invoice. The dashboard shows all three cases.

Agent 7 — Orchestrator, Approval & Audit

Live. A LangGraph-style state machine enforces legal transitions, the $4,000 approval gate and the audit trail. `POST /api/workflow/e2e-simulation` runs the eight-stage acceptance test against INV-2026-0891: ingest $4,850, age it, classify the reply, extract facts and check the date, draft, block auto-send at the gate, reconcile a $2,500 partial deposit, resync the balance to $2,350 and append the audit record. The approval queue cards on the dashboard are a seeded visualisation; the buttons update local UI state, not the backend. The **real** approval endpoint is `POST /api/approvals/<task_id>/decision`.

Appendix C — Three-minute lightning version

1. **0:00** Campus: "Seven agents, one ring, WebGL, built in React Three Fiber." Walk once, press E, close the modal.
2. **0:30** Upload Bill → upload `INV-2026-8802.pdf` → "Text extraction, Pydantic validation, line items balance, 98.5%." (60 s)
3. **1:30** Agent 3 → Billing Dispute → "94% confident, therefore a human, not a robot." (30 s)
4. **2:00** Agent 4 → conflict conversation → "Tuesday vs Thursday; both statements surfaced, reminder paused." (30 s)
5. **2:30** Agent 7 → Execute Full Acceptance Test → "Eight stages, seven agents, gate blocked above $4,000, balance reconciled to $2,350." (30 s)
6. **3:00** Close.

Appendix D — Likely judge questions

"Is the 3D just a video?" No. It is a live WebGL canvas with a playable avatar, per-frame proximity checks and clickable agents. The labels are DOM overlays drawn by drei.

"Are the agents really LLMs?" The architecture is deterministic-first. Agents 1, 2, 6, 7 are deterministic engines. Agent 3 is a calibrated scorer. Agent 4 and 5 can call a hosted model through RunPod, but the model output is treated as an untrusted draft and is re-checked against deterministic facts. With no endpoint configured, the app uses the local deterministic path and says so.

"What stops the model from sending a wrong email?" Three things: the P0 date-vs-weekday guard, the prompt-injection screen, and the $4,000 approval gate in backend policy. A model cannot clear a block; only deterministic code and a human can.

"Why does it say RunPod not linked?" Because no serverless endpoint has been provisioned. The worker, Dockerfile and client are in the repository; the app degrades to local engines and reports the state instead of inventing a connection.

"Is the approval queue wired to the backend?" The acceptance test's gate is real backend policy. The two cards in the Agent 7 dashboard are seeded demo data; the backend decision endpoint exists at `/api/approvals/<task_id>/decision`.

"Does the upload survive a restart?" The Render instance keeps the store for the life of the process. A restart reseeds the demo data. Uploaded invoices are visible in the ledger and the Executive Overview for the current session.